# Instalação e operação

Documentacao tecnica - Manual Operacional Digital

## Requisitos

- Node.js compatível com o Vite 8.
- npm.
- PostgreSQL.
- Opcional: projeto e bucket no Supabase Storage.
- `psql` disponível para aplicar o schema.

## Banco de dados

Crie o banco e execute o schema. O arquivo é idempotente e também serve como migração dos ambientes existentes.

```
$env:DATABASE_URL='postgres://postgres:senha@localhost:5432/manual_operacional'
psql "$env:DATABASE_URL" -v ON_ERROR_STOP=1 -f backend/sql/schema.sql
```

Conta inicial criada pelo schema:

- Identificador: `admin`
- Senha: `admin123`

Troque essa senha após o primeiro acesso, especialmente em produção.

## Backend

```
cd backend
Copy-Item .env.example .env
npm install
npm run dev
```

Variáveis:

| Variável | Obrigatória | Descrição |
|---|---|---|
| `PORT` | Não | Porta HTTP; padrão esperado: `4000` |
| `DATABASE_URL` | Sim em produção | String de conexão PostgreSQL |
| `JWT_SECRET` | Sim | Segredo de assinatura dos tokens |
| `FRONTEND_URL` | Sim em produção | Origem permitida pelo CORS |
| `SUPABASE_URL` | Não | URL do projeto Supabase |
| `SUPABASE_SERVICE_ROLE_KEY` | Não | Chave privada usada somente no backend |
| `SUPABASE_BUCKET` | Não | Bucket; padrão `manuals` |
| `DEV_DEMO_MODE` | Não | Usa memória quando o banco estiver indisponível |

Sem Supabase, os arquivos são salvos em `backend/uploads`. Nunca exponha a chave `SUPABASE_SERVICE_ROLE_KEY` no frontend.

# Frontend

```
cd frontend
Copy-Item .env.example .env
npm install
npm run dev
```

Variável disponível:

| Variável | Descrição |
| --- | --- |
| `VITE_API_URL` | URL pública do backend; padrão local `http://localhost:4000` |

Acesse `http://localhost:5173`.

# Execução de produção

```
cd frontend
npm run build

cd ../backend
npm start
```

O diretório `frontend/dist` deve ser servido por um host estático ou proxy reverso. O backend não serve automaticamente o build do frontend.

# Verificações

```
cd backend
npm test
node --check src/app.js

cd ../frontend
npm run build
```

Teste de saúde:

```
Invoke-RestMethod http://localhost:4000/health
```

Resposta esperada:

```
{
  "status": "ok",
  "service": "Manual Operacional Digital API"
}
```

# Modo demonstração

Com `DEV_DEMO_MODE=true`, erros de indisponibilidade do PostgreSQL habilitam dados em memória. Esse modo é útil somente para desenvolvimento:

- dados desaparecem ao reiniciar;
- não substitui migrações reais;
- alguns recursos avançados dependem do PostgreSQL;
- não deve ser habilitado em produção.

# Diagnóstico rápido

- 401: token ausente, inválido ou expirado.
- 403: perfil sem autorização.
- Banco indisponível: confirme `DATABASE_URL` e se o PostgreSQL está ativo.

- CORS: confirme `FRONTEND_URL` exatamente com protocolo e porta.
- Upload local não abre: confira permissões e a pasta `backend/uploads`.
- PDF não atualiza progresso: confirme que o backend foi reiniciado e o `schema.sql` aplicado.